

A1Q2

Lily Li

2025-05-15

Q2

```
library(cancensus)
```

```
## Census data is currently stored temporarily.
```

```
##
```

```
## In order to speed up performance, reduce API quota usage, and reduce unnecessary network calls, please
```

```
## This will add your cache directory as environment variable to your .Renviron to be used across sessions
```

```
library(readr)
```

```
library(ggplot2)
```

```
library(dplyr)
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
## filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
## intersect, setdiff, setequal, union
```

```
all_regions = list_census_regions("CA21") # 2021 census
```

```
## Querying CensusMapper API for regions data...
```

```
rural_on_regions = subset(all_regions,
```

```
  level=="CSD" & # census subdivisions
```

```
  PR_UID == 35 & # in Ontario
```

```
  is.na(CMA_UID)) # not inside a metropolitan area
```

```
csd_list = list(CSD = rural_on_regions$region)
```

```
census_data = get_census(dataset = "CA21",
```

```
  regions = csd_list,
```

```
  level = "CSD")
```

```
## Census data is currently stored temporarily.
##
## In order to speed up performance, reduce API quota usage, and reduce unnecessary network calls, please
## This will add your cache directory as environment variable to your .Renviron to be used across sessions

## Census data is currently stored temporarily.
##
## In order to speed up performance, reduce API quota usage, and reduce unnecessary network calls, please
## This will add your cache directory as environment variable to your .Renviron to be used across sessions

## Querying CensusMapper API...

## Downloading: 11 kB      Downloading: 11 kB      Downloading: 11 kB      Downloading: 11 kB
```

```
names(census_data)[4] = "Area"
head(census_data)
```

```
## # A tibble: 6 x 10
##   GeoUID Type 'Region Name' Area Population Dwellings Households CD_UID PR_UID
##   <chr> <fct> <fct>      <dbl>      <dbl>      <dbl>      <dbl> <chr> <chr>
## 1 35010~ CSD   South Glenga~ 605.      13330      5848      5431 3501 35
## 2 35010~ CSD   Akwesasne (P~ 11.8        0         0         0 3501 35
## 3 35010~ CSD   South Dundas~ 522.      11044      4821      4651 3501 35
## 4 35010~ CSD   North Dundas~ 502.      11304      4673      4537 3501 35
## 5 35010~ CSD   North Stormo~ 515.       7400      2949      2853 3501 35
## 6 35010~ CSD   North Glenga~ 643.      10144      4714      4422 3501 35
## # i 1 more variable: CMA_UID <chr>
```

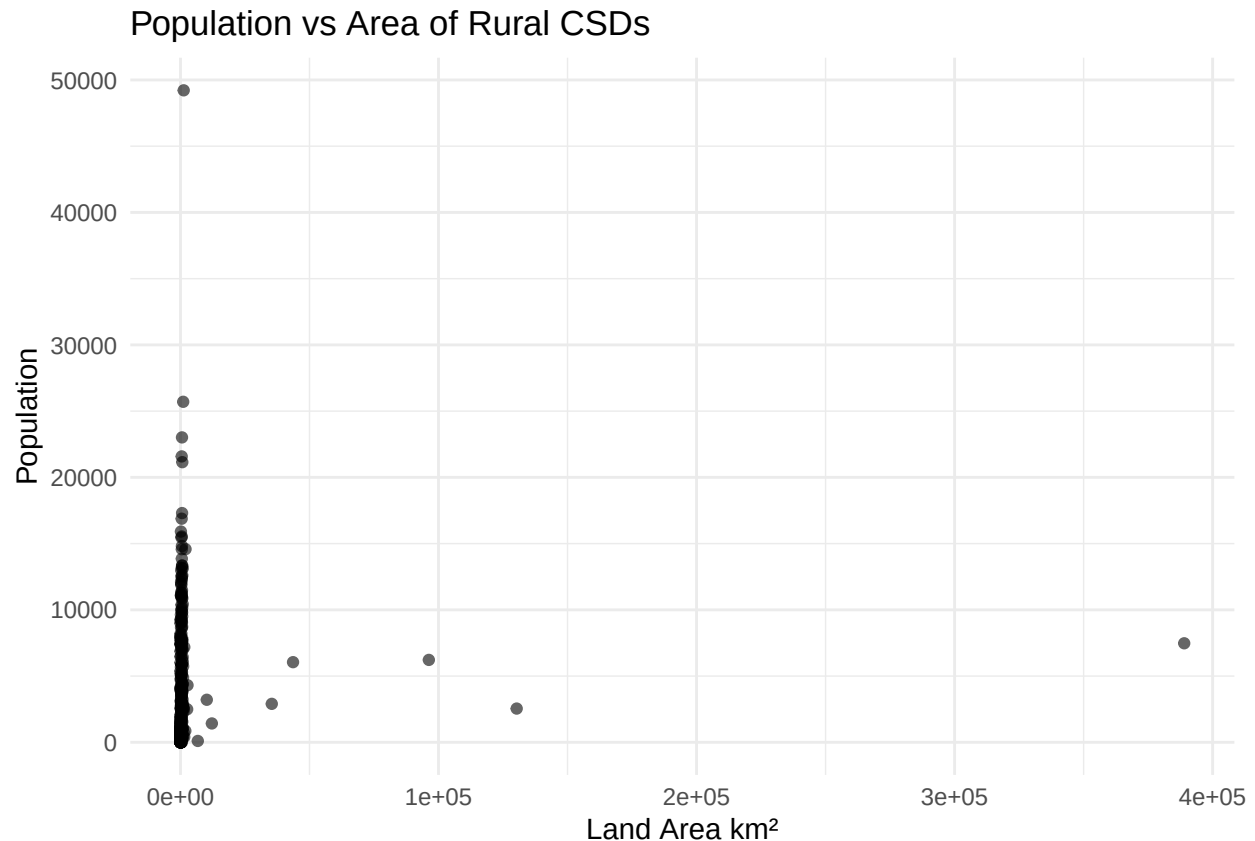
(b)

```
cor(census_data$Area, census_data$Population, use = "complete.obs")
```

```
## [1] 0.04925805
```

(c)

```
ggplot(census_data, aes(x = Area, y = Population)) + geom_point(alpha = 0.6) +
  labs(title = "Population vs Area of Rural CSDs",
       x = "Land Area km2", y = "Population") +
  theme_minimal()
```



From the scatterplot above, it is clear that most data points are densely clustered near the origin, while a few extreme outliers with very large land areas stretch the x-axis. This uneven spread makes it difficult to observe any meaningful relationship in the raw scale. Applying a **logarithmic transformation** to both area and population would help by **compressing the large values** and **spreading out the smaller, clustered points**, thereby making the overall relationship more linear and likely increasing the correlation coefficient.

(d)

```
transformations = list(log = function(x) log(x + 1), sqrt = function(x) sqrt(x),
  inv = function(x) -1 / (x + 1), square = function(x) x^2,
  cube = function(x) x^3)

results = data.frame(Area_transformed = character(0),
  Population_transformed = character(0),
  Correlation = numeric(0))
for(area_name in names(transformations)) {
  for(pop_name in names(transformations)) {
    try({
      tx = transformations[[area_name]](census_data$Area)
      ty = transformations[[pop_name]](census_data$Population)
      cor_val = cor(tx, ty, use = "complete.obs")
      results = rbind(results, data.frame(Area_transformed = area_name,
        Population_transformed = pop_name,
```

```

Correlation = cor_val))
  }, silent = TRUE)
}
}

results <- results %>% mutate(abs_corr = abs(Correlation)) %>%
  arrange(desc(abs_corr))

print(results)

```

##	Area_transformed	Population_transformed	Correlation	abs_corr
## 1	log	log	0.509847163	0.509847163
## 2	log	sqrt	0.503416502	0.503416502
## 3	inv	log	0.362663384	0.362663384
## 4	inv	sqrt	0.310712662	0.310712662
## 5	log	inv	0.234665618	0.234665618
## 6	inv	inv	0.206517691	0.206517691
## 7	log	square	0.200380928	0.200380928
## 8	sqrt	sqrt	0.196841216	0.196841216
## 9	sqrt	log	0.195437085	0.195437085
## 10	log	cube	0.106565914	0.106565914
## 11	inv	square	0.101715829	0.101715829
## 12	sqrt	inv	0.077845863	0.077845863
## 13	sqrt	square	0.061116487	0.061116487
## 14	square	sqrt	0.058766896	0.058766896
## 15	cube	sqrt	0.056050079	0.056050079
## 16	square	log	0.051691549	0.051691549
## 17	cube	log	0.047675596	0.047675596
## 18	inv	cube	0.046371042	0.046371042
## 19	sqrt	cube	0.031654962	0.031654962
## 20	square	inv	0.014790282	0.014790282
## 21	cube	inv	0.013134870	0.013134870
## 22	cube	square	0.006959442	0.006959442
## 23	square	square	0.006117176	0.006117176
## 24	square	cube	-0.002833442	0.002833442
## 25	cube	cube	-0.002225491	0.002225491

Among all tested combinations of power transformations, the **logarithmic transformation applied to both area and population** resulted in the strongest linear correlation. The corresponding Pearson correlation coefficient was **0.5098**, indicating a moderate positive linear relationship after the log-log transformation.